AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

From a paper algorithm to Python

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

Name the quantities. Follow the order. Compare the output.

Translate a small written calculation into Python without confusing assignment, display and measurement.

Keep one habit: Packing trace → complete seed script → five order/boundary tests → selected transfer → portfolio record.

MAIN LESSON · SECTION 1

How does a paper rule become a saved Python script?

> Trace the values and statement order before running the file.

Your learning goals

I can translate a short paper calculation into a Python script.

I can predict and explain the order of executed statements.

I can keep expected and observed **output** separate when checking a **program**.

Recall the equal-group calculation

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.
- Four packs contain six counters each. What steps find and show the total?
- After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.
- Four packs contain six counters each. What steps find and show the total?

> **My starting explanation**

Your end goal

Trace from a paper **algorithm** to python and explain the changed **program** before running it.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Your learning target

Remember before reading

This week you will practise:

Translate a short paper calculation into a Python script

Predict and explain the order of executed statements

Keep expected and observed **output** separate when checking a **program**
</details>

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

Start with four packs of six counters

> Write, save and run a small complete calculation before a larger product.

The input and task contract

In Grade 6 you traced rules on paper. Now you will write a small Python **program** and check what it actually does. Begin with a counter-packing calculation, not the whole activity adviser.

The requirement is to find the total counters in equal packs and display that total. For this first **program**, the supplied quantities are whole, nonnegative numbers; it does not yet validate typed **input**.

Use the prepared editor or label a paper trace

Use a teacher-prepared Python 3 editor that can run a saved .py file. Open a new file called counters.py, type the code exactly as shown in the next section, save it, and use the editor’s Run command.

Read the **output** area separately from the source editor. No account, AI tool, package installation or personal data is needed. If a device is unavailable, trace on paper and label the result predicted rather than executed.

Your first explanation

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

Assign the inputs, calculate, then display

> Assignment stores a value; print produces the output.

Map paper actions to statements

A calculation from paper to output

PAPER STEP	PYTHON INSTRUCTION	EFFECT
Record pack count	packs = 4	Name holds 4
Multiply two quantities	total = packs * per_pack	Name holds product
Show the answer	print(total)	Displays the value

Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

Complete four-line script

```
packs = 4
per_pack = 6
total = packs * per_pack
print(total)
```

Trace stored values and displayed output

Statement	Value after it runs	Visible output
packs = 4	packs holds 4	Nothing yet
per_pack = 6	per_pack holds 6	Nothing yet
total = packs * per_pack	total holds 24	Nothing yet
print(total)	Stored values unchanged	24 on its own line

Read the syntax and order

Read the script from top to bottom. The equals sign assigns the value on its right to the name on its left. The star means multiplication. print(total) displays the value of total; it does not display the letters in the name.

Type ordinary straight characters and keep the parentheses. The Python prompt symbols >>> are not part of this saved script.

Explain the deciding step

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare changed inputs without changing the rule

> Keep the packing task separate from the pictured spare-seed task.

Predict the supported change

- Make records for 4 packs of 6, then 5 packs of 6. Predict before running. Record actual **output**, any error and what changed. If working on paper, label both results as traces.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Predict the pictured variant

- Change: Change spare to 5 while keeping both tray **input**s fixed.

▼ **Check after predicting**

Prediction: The printed total becomes 29.

Reason: 4*6 remains 24; adding five gives 29.

▼ Earlier wording, alternatives and picture transcripts

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace a second complete procedure: trays plus a spare pile

> Every name corresponds to a supplied quantity.

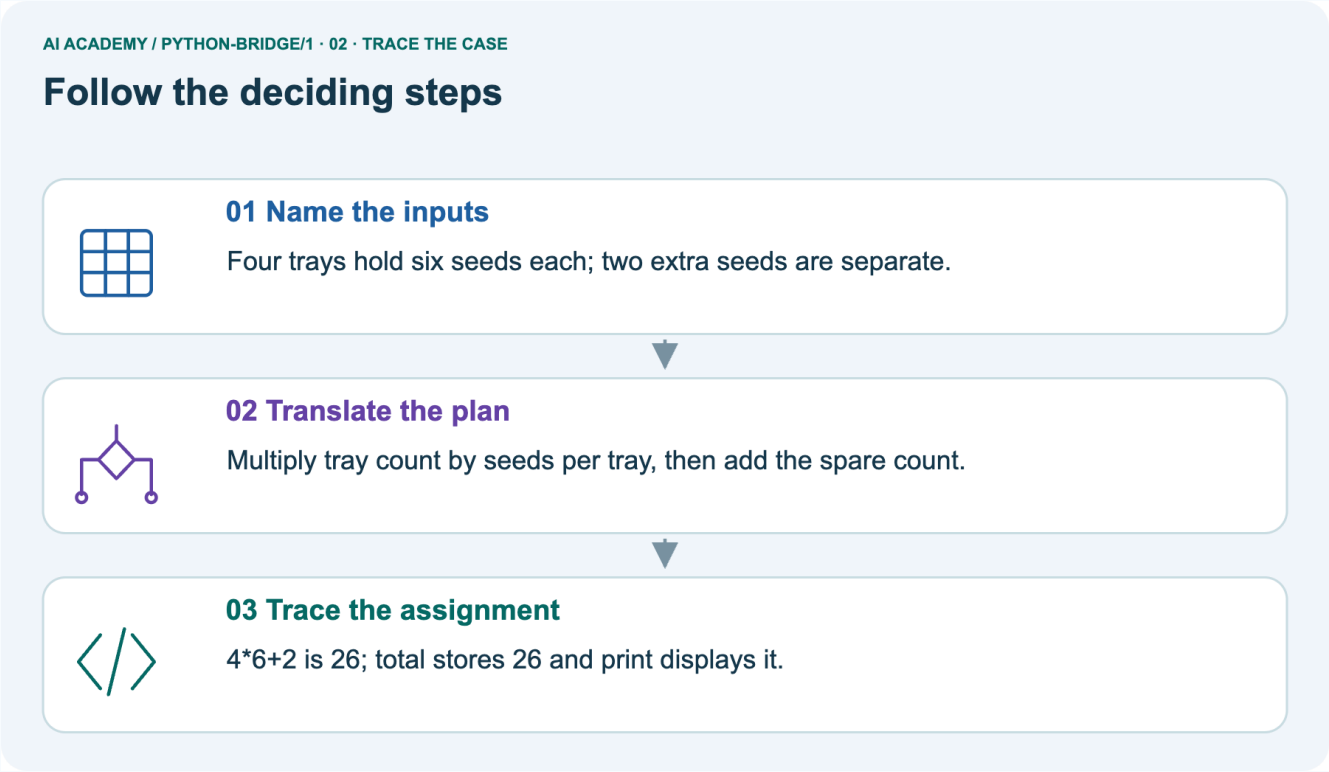
Read the complete seed task and inputs

Convert a seed-count **algorithm**. The paper plan counts seeds in identical trays, then adds a labelled spare tray.

Python 3: trays=4; per_tray=6; spare=2; total=trays*per_tray+spare; print(total). All quantities count seeds.

The seed task has four equal trays of six seeds plus two separate spare seeds. Multiply only the identical tray groups, then add the spare pile once: 4*6+2=26. Changing spare to 5 gives 29 without changing tray counts. The packing warm-up has no spare pile and gives 24. All quantities are supplied whole nonnegative counts; neither script validates arbitrary typed input or detects a physical miscount.

Trace the picture and deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Name the inputs	Four trays hold six seeds each; two extra seeds are separate.
2. Translate the plan	Multiply tray count by seeds per tray, then add the spare count.
3. Trace the assignment	4*6+2 is 26; total stores 26 and print displays it.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Python follows the arithmetic instructions. This translation automates a declared counting rule; it does not train a model from examples.

Read the complete five-line seed script

```
trays = 4
per_tray = 6
spare = 2
total = trays * per_tray + spare
print(total)
```

Predict changing only the spare pile

- Change one thing: Change spare to 5 while keeping both tray **input**s fixed. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting

AI ACADEMY / PYTHON-BRIDGE/1 · 04 · COMPARE THE CASES

Change one thing. Trace what follows.

Original case

4*6+2 is 26; total stores 26 and print displays it.

Changed condition

Change spare to 5 while keeping both tray inputs fixed.

Worked changed-case result

The printed total becomes 29.
Why: 4*6 remains 24; adding five gives 29.

Follow the labelled picture with the matching explanation. Open the vector diagram

State the counted-input limit

- The calculation assumes identical tray sizes and measured counts; it cannot detect a miscounted tray.

- Read the labelled picture: Match each paper quantity to its Python name. Why does the spare count get added once, and what prints when spare is five?

▼ Packing warm-up · Expected and observed record

After line	Names and values	Displayed output
packs = 4	packs is 4	Nothing yet

After line	Names and values	Displayed output
per_pack = 6	packs is 4; per_pack is 6	Nothing yet
total = packs * per_pack	total is 24	Nothing yet
print(total)	Values are unchanged	24

Write 24 as the expected **output** before running. Then copy the actual **output** into a separate column. A matching result supports this case, not every possible **input**.

A saved file may contain an old edit: save again and rerun the whole script. Do not type the expected number into the **output** record without running.

> **Why is there no displayed output after the third line? What changes if print(total) is placed first in a fresh run, before any assignment? Explain using which names exist.**

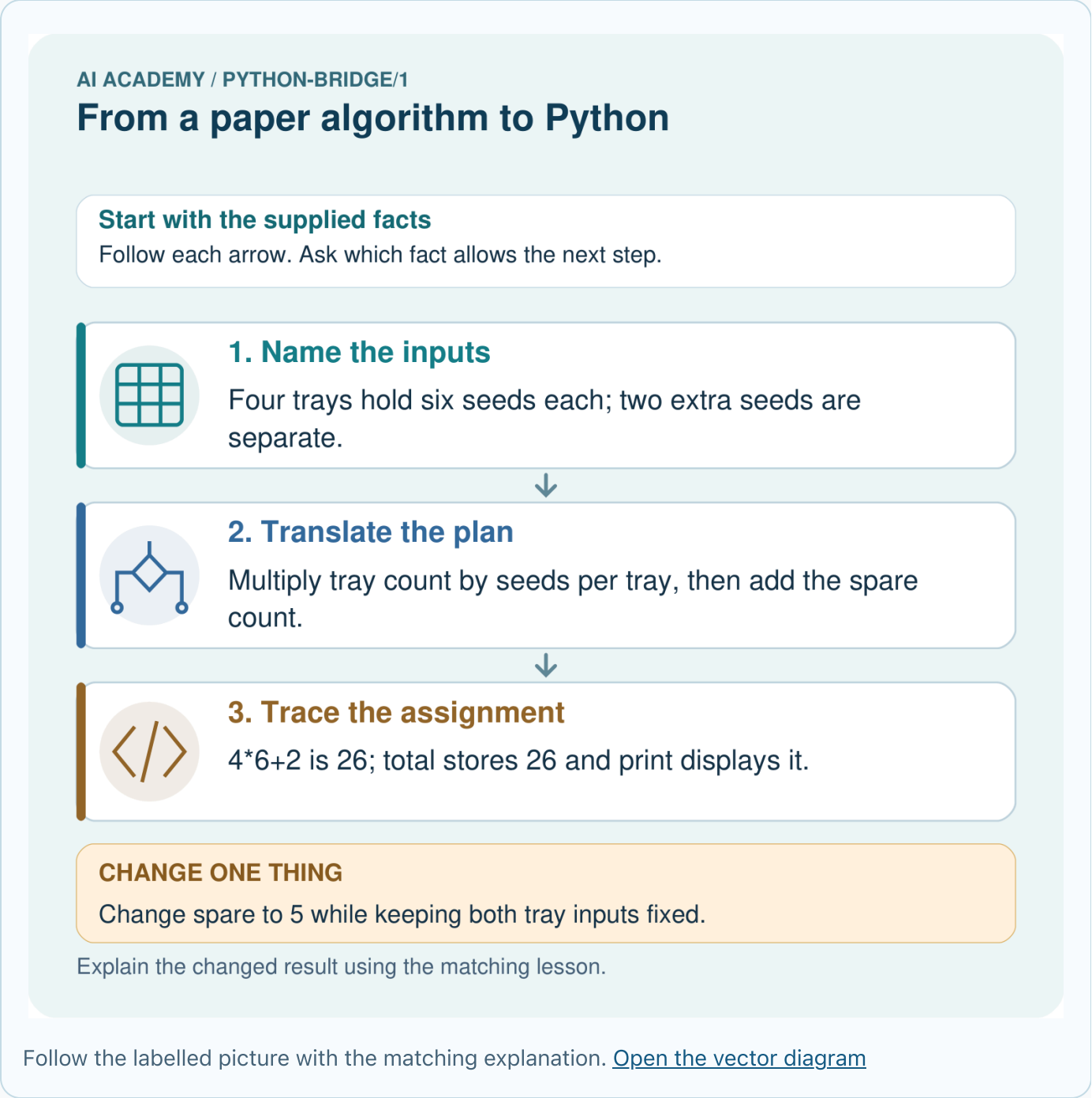
▼ Earlier wording, alternatives and picture transcripts

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: From a paper algorithm to Python

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Find a wrong rule or an early print statement

> A program can finish without satisfying the task.

Change packs, save and run the whole file

Change only packs to 5. Predict the result from five groups of six, save, then run the entire file again. The expected **output** is 30. Keep the original four-pack result beside the new one and label the changed **input**.

A comment can explain an instruction: Python ignores text after # outside a string. A comment does not change the calculation.

```
packs = 5  # five equal packs
per_pack = 6
total = packs * per_pack
print(total)
```

Inspect the addition-instead-of-multiplication error

A **program** can run successfully and still solve the wrong problem. Replacing * with + would display 11 for this **input**, but addition does not count five groups of six.

Use the requirement and a paper calculation to check meaning; the absence of an error message is not proof of correctness.

Predict reading total before it exists

- Move print(total) to the first line of a fresh script. Predict what happens and why. Put the lines back into an order that meets the requirement.

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Explain the packing script with less help

> Use one response record and label the help you used.

Explain each line

- Write the complete four-line script for 4 packs of 6. Beside each line explain what it does. Which line displays information?

Why is there no displayed output after the third line? What changes if print(total) is placed first in a fresh run, before any assignment? Explain using which names exist.

Explain your prediction to a partner before comparing results.

Complete W2 prediction and observation

Use workbook W2 for the connected example “From a paper **algorithm** to Python”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Check assignment versus display

- Use the worked case for From a paper **algorithm** to Python. Proposed explanation: “An equals sign displays the answer”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the pictured spare count

- Match each paper quantity to its Python name. Why does the spare count get added once, and what prints when spare is five?

My explanation using a pictured step and the given facts

Reserve the selected transfer task

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Predict then observe

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Check five complete packing and order cases

> Do not replace an observed error with the output you hoped to see.

All supplied test inputs

Complete fictional test	Prediction and what to record
A · packs 4, per_pack 6, multiply then print	24; record execution or paper trace separately.

Complete fictional test	Prediction and what to record
B · Change only packs to 5	30; save the edited script before rerunning.
C · Change only packs to 0	0; the stated empty-group boundary.
D · Fresh script begins print(total), no earlier total assignment	NameError before calculation; retain the actual error if run.
E · Replace multiplication with addition for packs 5/per_pack 6	11; successful execution but wrong equal-group procedure.

Predict, change one thing and preserve the result

1. Predict A/B before running the two supplied scripts. Then test C/D/E one at a time, keeping original and changed code versions. Run the entire saved file in a fresh script/namespace for D, because an old interactive total could hide the error. Record expected output, actual output/error, code change, save/run status and first mismatch. Paper traces are valid reasoning practice but do not establish editor/run skill.

   These tests apply a written rule, not model training. Keep the selected 3-packs-of-7 transfer code unused until the individual attempt; if seen in an earlier lesson, record exposure and use a fresh parallel input for assessment.

Original A/B complete scripts

```
packs = 4
per_pack = 6
total = packs * per_pack
print(total)
```

```
packs = 5  # five equal packs
per_pack = 6
total = packs * per_pack
print(total)
```

Record predictions and observations

Make records for 4 packs of 6, then 5 packs of 6. Predict before running. Record actual **output**, any error and what changed. If working on paper, label both results as traces.

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ **Earlier wording, alternatives and picture transcripts**

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: trace the selected packs script and input change

> Keep a first attempt before hints and distinguish prior exposure.

Selected complete task and scope

Predict the printed value. Change only packs to 5 and predict again. Explain which statements apply a **written rule**.

```
packs = 3
per_pack = 7
total = packs * per_pack
print(total)
```

Use this complete selected script and change only the stated input. Explain the displayed output and written rule. If you already saw these exact values in earlier lessons, label that exposure; a teacher can supply different unseen nonnegative counts for a fresh check. The earlier shelf task below is additional practice with a different changed-input question.

Your response and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Additional practice · Earlier shelf transfer

A fictional shelf has 3 trays of 7 tiles each. Write a four-line Python script using trays, per_tray and tiles. Predict its **output**. Then change only trays to 0 and predict again. Explain why adding the two quantities is wrong.

> **My independent procedure, result and limitation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Trace variables and a change

Attempt this without hints

Use feedback to improve

MAIN LESSON · SECTION 11

Keep a small Python-readiness evidence sheet

> Save source code and observed output as different artefacts.

This week’s contribution

Add a labelled from a paper **algorithm** to python **evidence** sheet to Python readiness portfolio. Show how you can translate a short paper calculation into a python script.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project checkpoint and evidence

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

MAIN LESSON · SECTION 16

Finish: explain the inputs, order and displayed result

> A saved script, a correct trace and an actual run are separate evidence.

Explain without notes

- Close your notes. Explain this goal in your own words: Translate a short paper calculation into a Python script

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · The words used in this procedure**

REFERENCE · SECTION 5

Reference · The words used in this procedure

> Explain each term using a particular code line.

Vocabulary and response

Word	Meaning
Algorithm	An ordered procedure for solving a stated problem
Program	Instructions written in a language a computer can execute
Interpreter	Software that executes Python instructions
Assignment	Binding a name to the value of an expression
Output	Information a program displays or returns

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain, apply and challenge

REFERENCE · SECTION 10

Extra challenge · Explain, apply and challenge

> Choose the next useful task after supported practice.

Choose a challenge

- Write the complete four-line script for 4 packs of 6. Beside each line explain what it does. Which line displays information?

- Move print(total) to the first line of a fresh script. Predict what happens and why. Put the lines back into an order that meets the requirement.

- Explain the difference between a successful run and a correct calculation. State one limit of this **program** and what you would keep so another person can check your work.

Explain which evidence supports your conclusion and which claim remains unproved.

▼ Earlier wording, alternatives and picture transcripts

Explain → apply → challenge

▼ Reference · Inspect your evidence

REFERENCE · SECTION 12

Reference · Inspect your evidence

> Use the record after practice rather than revealing the independent answer.

Self-check and explanation

- Save the file with the **input** values, expected and observed **output**s, and a short explanation. This **program** follows a written multiplication rule.

  It has not trained a model, predicted somebody's preferences or checked that real packs contain the stated number. Its conclusion depends on the supplied data. Week 2 examines the values and types behind these names.

- Explain the difference between a successful run and a correct calculation. State one limit of this **program** and what you would keep so another person can check your work.

> **A script runs without an error and displays 11 for five packs of six. Is it correct for the stated requirement? Give the required result, the likely wrong operation and one check after repair. State whether your evidence came from a device run or a paper trace.**

My limited claim and a useful next check

▼ **Extra practice · A complete trace**

REFERENCE · SECTION 13

Extra practice · A complete trace

> Predict before opening the supplied trace.

Explain the input and procedure

- Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Check the complete trace

A calculation from paper to output

PAPER STEP	PYTHON INSTRUCTION	EFFECT
Record pack count	packs = 4	Name holds 4
Multiply two quantities	total = packs * per_pack	Name holds product
Show the answer	print(total)	Displays the value

Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

▼ Earlier wording, alternatives and picture transcripts

Complete evidence record

▼ Reference · Earlier route checklist

REFERENCE · SECTION 15

Reference · Earlier route checklist

> Keep one main lesson route and use this as a reminder.

Checklist and first prediction

By the end, I can	How I will show it
Translate a short paper calculation into a Python script	Explain the deciding step, show a trace or test, and state one limit.
Predict and explain the order of executed statements	Explain the deciding step, show a trace or test, and state one limit.
Keep expected and observed output separate when checking a program	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: Convert a seed-count **algorithm**. The paper plan counts seeds in identical trays, then adds a labelled spare tray.

▼ Extra practice · Predict, check and repair

REFERENCE · SECTION 17

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- Convert a seed-count **algorithm**. The paper plan counts seeds in identical trays, then adds a labelled spare tray.
- Which labelled step matches this detail? 4*6+2 is 26; total stores 26 and print displays it.
- 1. Trace the **assignment**
- 2. Name the **input**s
- 3. Translate the plan
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Trace the **assignment** 4*6+2 is 26; total stores 26 and print displays it. Python follows the arithmetic instructions.

This translation automates a declared counting rule; it does not train a model from examples.

Changed-case question and hint

- Changed-case practice: Change spare to 5 while keeping both tray **input**s fixed. Which conclusion is supported by the supplied facts?

- 1. The printed total becomes 29.
- 2. We can decide the result without examining the changed **input** or the procedure.
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: The printed total becomes 29. 4*6 remains 24; adding five gives 29.

Repair and keep the record

Review Debugging Strategies (ai-2/6). Goal: Design revealing tests for a function. Short practice: Which test most directly isolates the handling of loose books, and what should it produce?.

Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

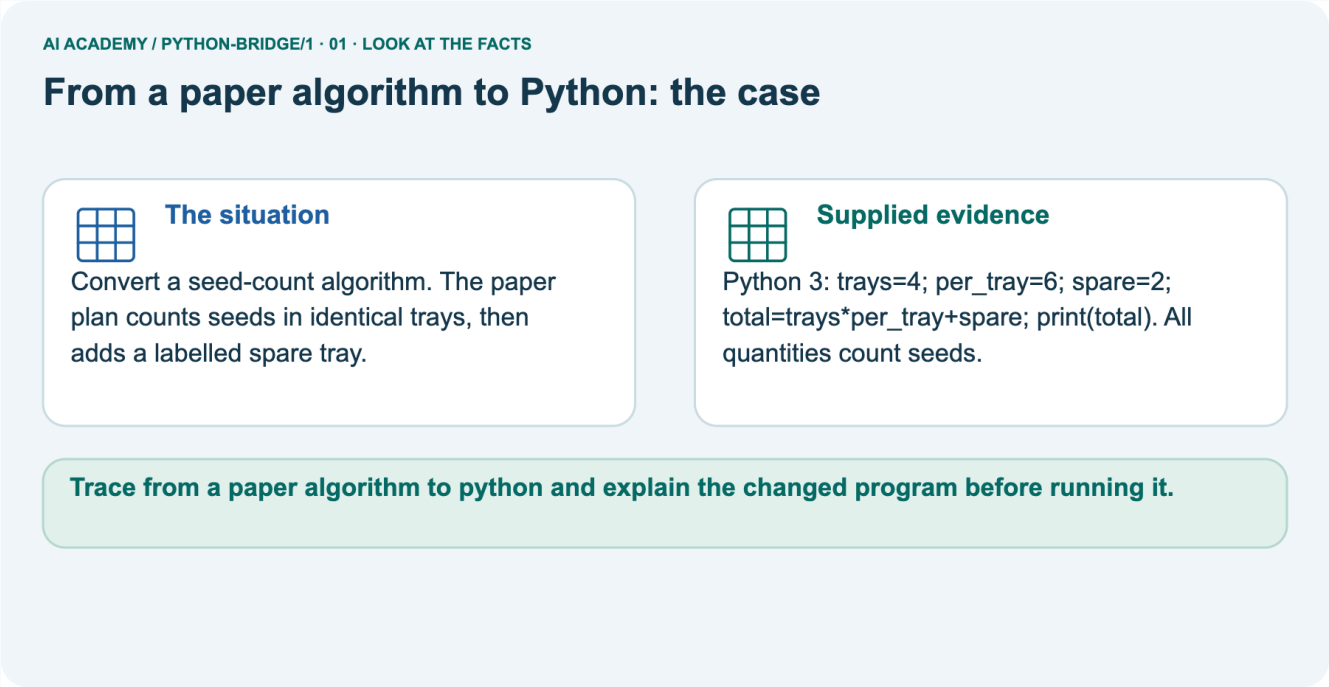
▼ Reference · The six-picture journey

REFERENCE · SECTION 18

Reference · The six-picture journey

> Choose the picture needed for the present question.

01. From a paper algorithm to Python: the case



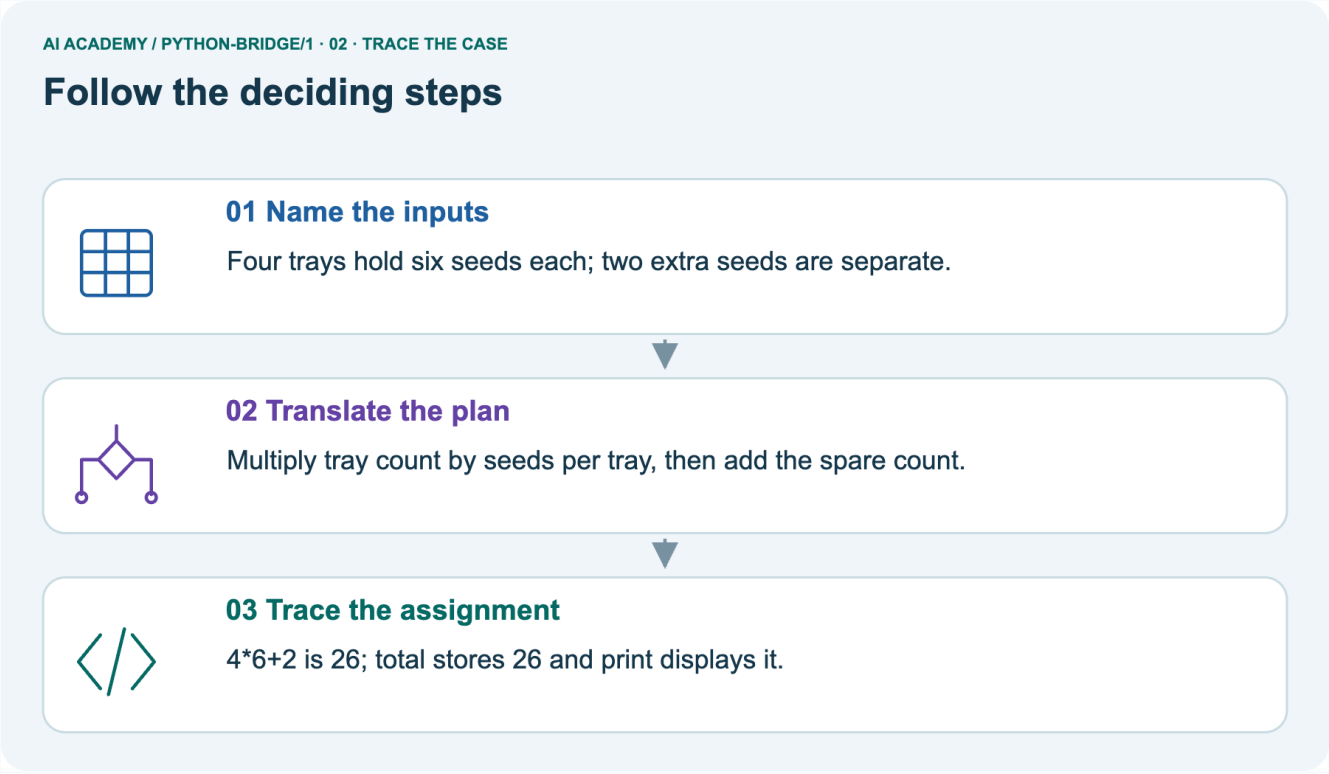
Follow the labelled picture with the matching explanation. Open the vector diagram

Convert a seed-count **algorithm**. The paper plan counts seeds in identical trays, then adds a labelled spare tray. Supplied facts: Python 3: trays=4; per_tray=6; spare=2; total=trays*per_tray+spare; print(total).

All quantities count seeds.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Name the **input**s: Four trays hold six seeds each; two extra seeds are separate. → Translate the plan: Multiply tray count by seeds per tray, then add the spare count. → Trace the **assignment**: 4*6+2 is 26; total stores 26 and print displays it.

Point to each numbered stage and explain which recorded **input** it uses.

03. A calculation from paper to output

AI ACADEMY / PYTHON-BRIDGE/1 · 03 · INSPECT THE LESSON MODEL

A calculation from paper to output

Paper step	Python instruction	Effect
Record pack count	packs = 4	Name holds 4
Multiply two quantities	total = packs * per_pack	Name holds product
Show the answer	print(total)	Displays the value

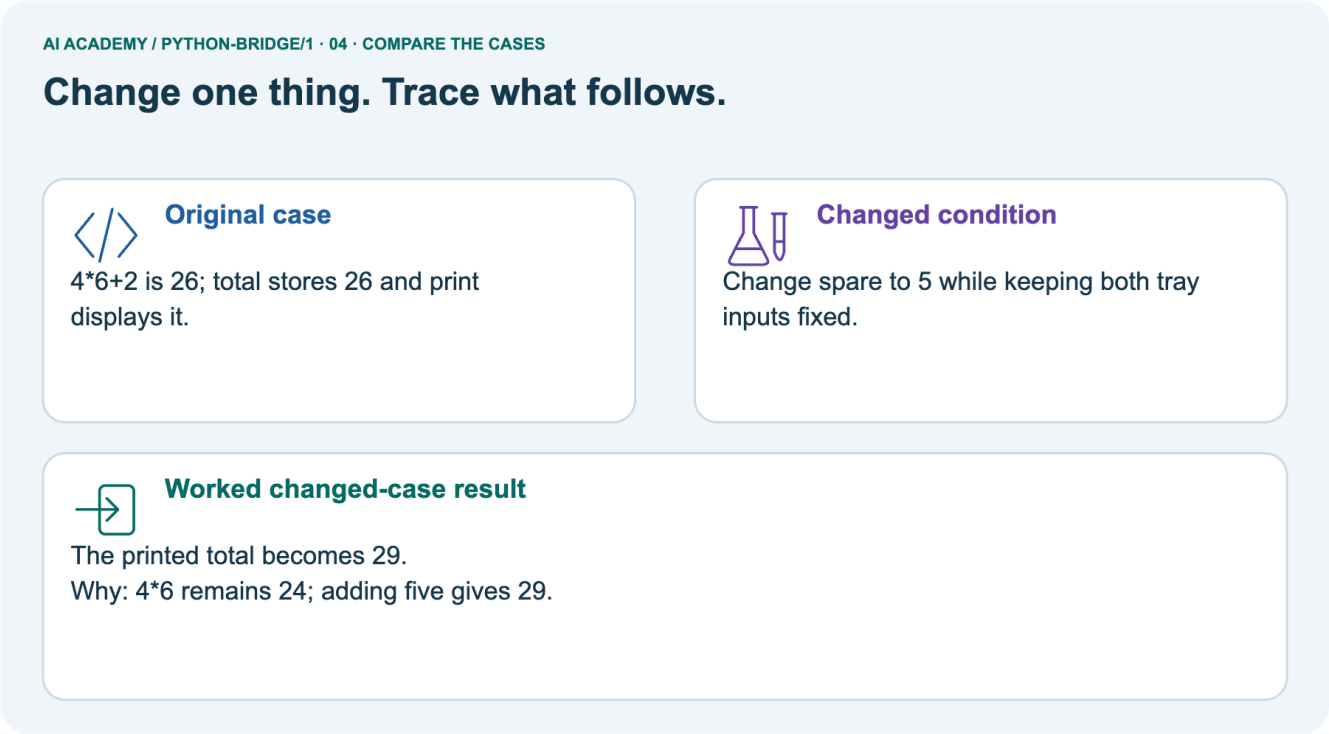
Assign per_pack before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

Follow the labelled picture with the matching explanation. Open the vector diagram

A calculation from paper to **output**. Record pack count | packs = 4 | Name holds 4. Multiply two quantities | total = packs * per_pack | Name holds product. Show the answer | print(total) | Displays the value

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



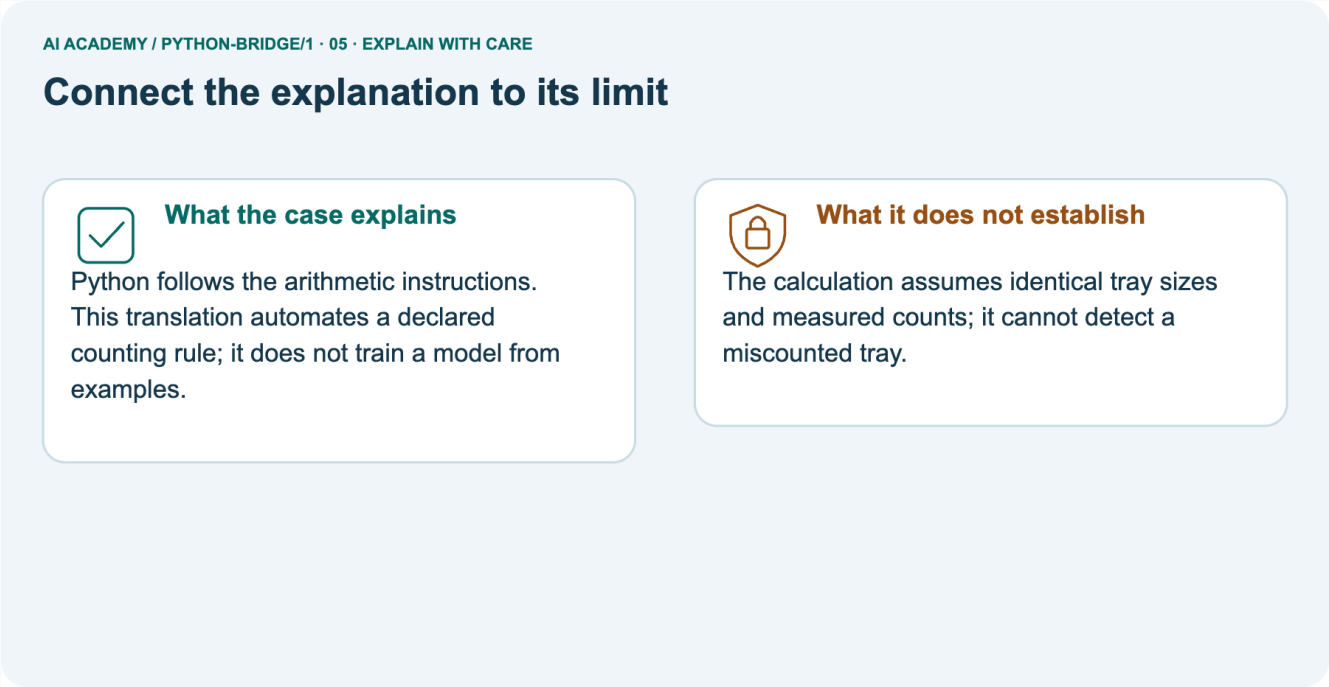
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: 4*6+2 is 26; total stores 26 and print displays it. Changed condition: Change spare to 5 while keeping both tray **input**s fixed. Expected result: The printed total becomes 29.

Reason: 4*6 remains 24; adding five gives 29.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



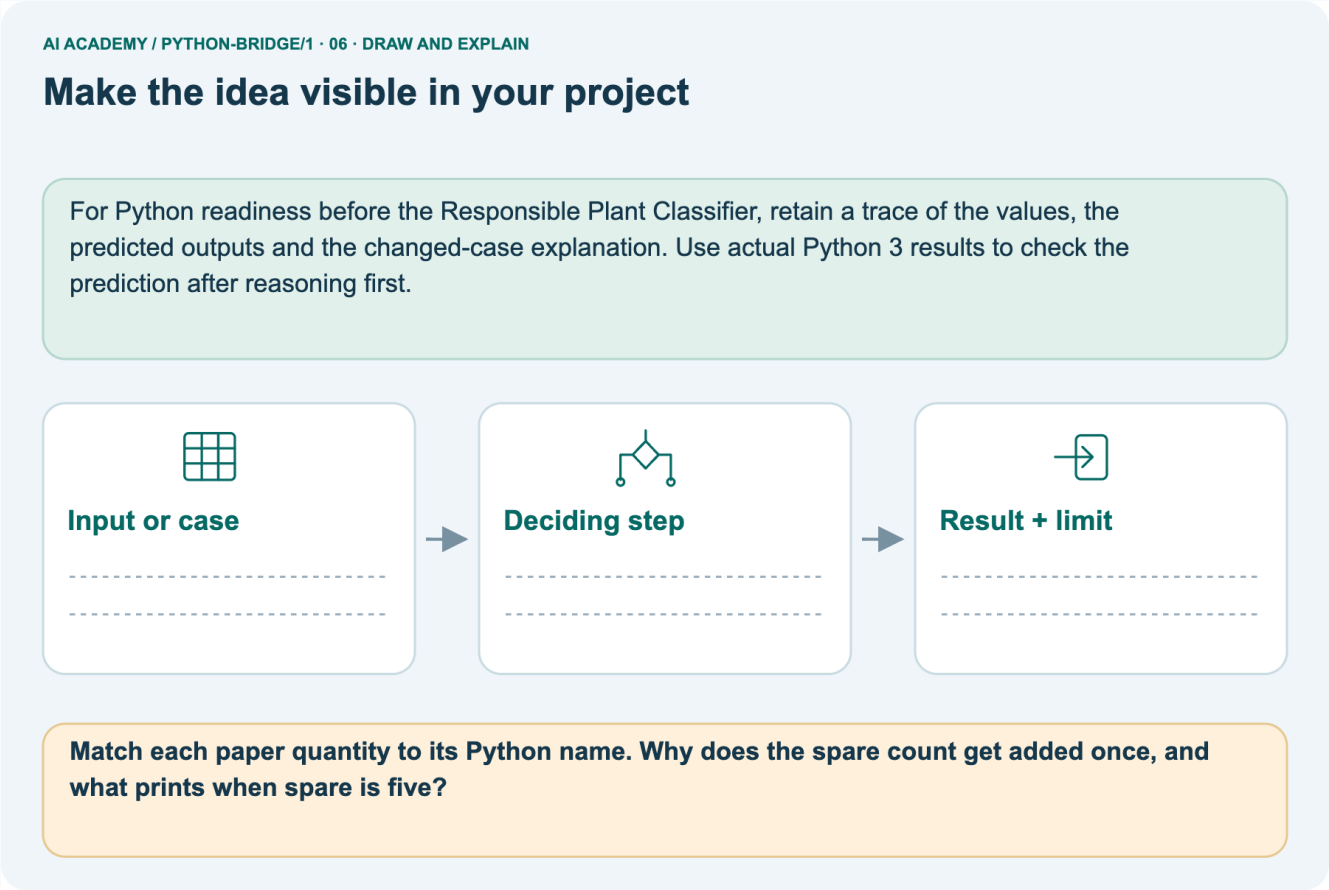
Follow the labelled picture with the matching explanation. Open the vector diagram

Python follows the arithmetic instructions. This translation automates a declared counting rule; it does not train a model from examples.

Limit: The calculation assumes identical tray sizes and measured counts; it cannot detect a miscounted tray.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Match each paper quantity to its Python name. Why does the spare count get added once, and what prints when spare is five?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. From a paper algorithm to Python: the case

02. Follow the deciding steps

03. A calculation from paper to output

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.